



Chapter 1

Python Functions

অধ্যায়-১: পাইথন ফাংশনস

Basic Application Development Using Python | Diploma in Engineering, 3CST

Md. Ruhul Amin • Junior Instructor, CST • Kushtia Government Polytechnic Institute



Chapter Roadmap

এই অধ্যায়ে আমরা কী শিখব



1.0–1.1 Function

ভূমিকা ও ফাংশন



1.2 Library vs User-defined

লাইব্রেরি ও ইউজার ডিফাইন



1.3 Calling & Return

ফাংশন কল ও রিটার্ন



1.4 Pass by Value / Reference

পাস বাই ভ্যালু / রেফারেন্স



1.5 Arguments

আরগুমেন্টের প্রকারভেদ



1.6 Date & Time

তারিখ ও সময় ফাংশন



1.7 Programs

১৫টি প্রোগ্রাম



Recap & Practice

রিক্যাপ ও অনুশীলন



1.0 Introduction

ভূমিকা: ফাংশন কেন দরকার?



Reusable Code

একই কোড বারবার ব্যবহার



Clean Structure

প্রোগ্রাম গোছানো থাকে



Saves Time

সময় বাঁচে

def = keyword to declare a function in Python | পাইথনে ফাংশন ডিক্লেয়ার করতে def ব্যবহার হয়



1.1 What is a Function?

ফাংশন কী?

A named block of organized, reusable code

নির্দিষ্ট নামের, গোছানো ও পুনঃব্যবহারযোগ্য কোড ব্লক

Easier debugging, testing, maintenance

ডিবাগিং, টেস্টিং, মেইনটেন্যান্স সহজ

Identified by () after its name

নামের শেষে এক জোড়া প্রথম বন্ধনী ()





Function Syntax & Rules

ফাংশন তৈরির সিনট্যাক্স

```
def function_name(parameters):  
    # function body  
    return value
```



Tip: no return needed if nothing to send back

কিছু রিটার্ন না করলে return না লিখলেও চলে

1. Start with def keyword

def দিয়ে শুরু করতে হয়

2. Meaningful name + ()

অর্থবোধক নাম ও প্যারামিটার

3. Colon (:) after ()

বন্ধনীর পর কোলন

4. return sends the result

return ফলাফল ফেরত দেয় (ঐচ্ছিক)



Anatomy of a Function

একটি ফাংশনের গঠন

```
def add_fun(a, b):  
    c = a + b  
    return c  
  
print("Addition A & B =", add_fun(15, 16))
```

Output:
Addition A & B = 31

Function name ফাংশনের নাম

add_fun

Parameters প্যারামিটার

a, b

Arguments আরগুমেন্ট

15, 16

Function calling ফাংশন কল

add_fun(15, 16)



Calling a Function Many Times

একটি ফাংশন বারবার কল

```
def add_fun(a, b):  
    c = a + b  
    return c  
  
print("Addition A & B =", add_fun(15, 16))  
print("Addition A & B =", add_fun(25, 16))  
print("Addition A & B =", add_fun(25, 26))
```

Output

```
Addition A & B = 31  
Addition A & B = 41  
Addition A & B = 51
```



Write once, use unlimited times

একবার লিখে অসংখ্যবার কল করা যায়



Types of Functions

ফাংশনের প্রকারভেদ



1. Standard Library / Built-in

ডেভেলপার কর্তৃক আগে থেকে দেওয়া।

যেমন:

print(),	input(),	int(),	float(),
sqrt(),	sum(),	max(),	min(),
len(),	sorted(),	count()	



2. User-defined

প্রোগ্রামার নিজের প্রয়োজনে তৈরি করে।

যেমন:

add_fun(), show(), calc()



Advantages of Functions

ফাংশন ব্যবহারের সুবিধা



No code repetition

একই কোড বারবার লিখতে হয় না



Smaller programs

প্রোগ্রাম আকারে ছোট হয়



Easy to understand

সহজে বুঝা যায়



Fast development

দ্রুত অ্যাপ্লিকেশন ডেভেলপ



Easy debug & test

ডিবাগিং, টেস্টিং, আপডেট সহজ



Reusable library

লাইব্রেরিতে জমা রেখে পুনঃব্যবহার



1.2 Library vs User-defined Function

লাইব্রেরি ও ইউজার ডিফাইন ফাংশনের তুলনা

Library Function / লাইব্রেরি	User-defined / ইউজার ডিফাইন
Comes with the interpreter / কম্পাইলারের সাথে দেওয়া থাকে	Created by the user as needed / ইউজার প্রয়োজনে তৈরি করে
Made by Python developers / ডেভেলপার তৈরি	Made by the programmer / প্রোগ্রামার তৈরি
Ready to use for coding ease / আগে থেকে দেওয়া	User creates on his own / নিজস্ব প্রয়োজনে
Works as soon as it is called / কল করলেই কাজ করে	Works when called after it is defined / ডিফাইনের পর কল করলে
Named by developers / নামকরণ ডেভেলপারের	Named by the programmer / নামকরণ প্রোগ্রামারের
max(), min(), sum(), len(), print(), int(), input()	show(), circlearea(), f2c(), calc(), add_fun()



Built-in Functions (1/4)

পাইথনের বিল্ট-ইন ফাংশন তালিকা

#	Function	Description
1	abs()	Absolute value of a number
2	all()	True if all elements are true
3	any()	True if any element is true
4	ascii()	Printable string representation
5	bin()	Integer to binary string
6	bool()	Converts a value to boolean
7	bytearray()	Array of given byte size
8	bytes()	Immutable bytes object
9	callable()	Checks if object is callable

#	Function	Description
10	chr()	Character from an integer
11	classmethod()	Class method for function
12	compile()	Returns a code object
13	complex()	Creates a complex number
14	delattr()	Deletes attribute from object
15	dict()	Creates a dictionary
16	dir()	Attributes of an object
17	divmod()	Tuple of quotient and remainder



Built-in Functions (2/4)

পাইথনের বিল্ট-ইন ফাংশন তালিকা

#	Function	Description
18	enumerate()	Returns an enumerate object
19	eval()	Runs code within program
20	exec()	Executes dynamic program
21	filter()	Iterator of elements which are true
22	float()	Floating point number
23	format()	Formatted value
24	frozenset()	Immutable frozenset
25	getattr()	Value of named attribute
26	globals()	Dict of global symbol table

#	Function	Description
27	hasattr()	Has named attribute?
28	hash()	Hash value of an object
29	help()	Built-in help system
30	hex()	Integer to hexadecimal
31	id()	Identity of an object
32	input()	Reads a line of string
33	int()	Integer from number or string
34	isinstance()	Instance of a class?



Built-in Functions (3/4)

পাইথনের বিল্ট-ইন ফাংশন তালিকা

#	Function	Description
35	issubclass()	Subclass of a class?
36	iter()	Iterator for an object
37	len()	Length of an object
38	list()	Creates a list
39	locals()	Dict of local symbol table
40	map()	Applies function to items
41	max()	Largest element
42	memoryview()	Memory view of an argument
43	min()	Smallest element

#	Function	Description
44	next()	Next element from iterator
45	object()	Creates featureless object
46	oct()	Integer to octal
47	open()	Returns a file object
48	ord()	Unicode code point of character
49	pow()	x to the power of y
50	print()	Prints the given object
51	property()	Returns a property attribute



Built-in Functions (4/4)

পাইথনের বিল্ট-ইন ফাংশন তালিকা

#	Function	Description
52	range()	Sequence of integers start to stop
53	repr()	Printable representation
54	reversed()	Reversed iterator
55	round()	Rounds float to n digits
56	set()	Returns a set
57	setattr()	Sets attribute value
58	slice()	Creates a slice object
59	sorted()	Sorted list from iterable
60	staticmethod()	Static method from function

#	Function	Description
61	str()	String representation
62	sum()	Adds items of an iterable
63	super()	Refer to parent class
64	tuple()	Creates a tuple
65	type()	Type of an object
66	vars()	__dict__ of a class
67	zip()	Iterator of tuples
68	__import__()	Called by import



1.3 Defining a Function

ফাংশন তৈরি (Function Definition)

```
def FunctionName(Parameters):  
    Statements  
    return [expression]
```

```
def mul(x, y):  
    return x * y  
print(mul(10, 15))
```

```
def div(x, y):  
    z = x / y  
    print(z)  
div(10, 5)
```



Syntax: def + name + () + :

সিনট্যাক্স: def, নাম, (), কোলন

Output
150

Output
2.0



Rules for Defining a Function

ফাংশন তৈরির নিয়মকানুন



Start with def

def দিয়ে শুরু; নাম
identifier-এর নিয়মে, _
দিয়ে শুরু করা যায়



Parameters inside ()

এক বা একাধিক
প্যারামিটার কমা দিয়ে, না-
ও থাকতে পারে



Colon (:) after ()

প্যারেনথেসিসের পর কোলন



Indented body + return

সব স্টেটমেন্ট ইনডেন্টেড;
return দিয়ে এক্সিকিউশন
শেষ



Examples: Without & With return

রিটার্ন ছাড়া ও রিটার্নসহ উদাহরণ

Without return রিটার্ন ছাড়া

```
def welcome():  
    print("Welcome to Computer Technology")  
welcome()  
welcome()
```

Output

```
Welcome to Computer Technology  
Welcome to Computer Technology
```

With return রিটার্নসহ

```
def square(x):  
    y = x * x  
    return y  
print(square(5) + square(5))
```

Output

```
50
```



Calling a Function

ফাংশন কলিং

```
FunctionName()  
FunctionName(arguments)
```

Defining alone does not run it

শুধু ডিফাইন করলে কাজ হয় না, কল করতে হয়

Call as many times as needed

যতবার ইচ্ছা কল করা যায়

No parameter প্যারামিটার ছাড়া

```
def show():  
    print("Do not adopt unfair means")  
show()
```

With parameters প্যারামিটারসহ

```
def add(a, b):  
    return (a + b)  
print(add(40, 17))  
print(add(278, 157))  
print(add(35, 95))
```

Output

```
57    435    130
```



Return of a Function

ফাংশনের রিটার্ন

```
def my_func1():  
    print("Hello World")  
    return None  
def my_func2():  
    print("Hello World")  
    return  
def my_func3():  
    print("Hello World")
```

```
def add(a, b, c):  
    return (a + b + c)  
temp = add(10, 25, 30)  
print(temp)
```

Output
65

Syntax: return or return (returnValue) | মান না দিলে None রিটার্ন হয়



Role of the return Statement

রিটার্ন স্টেটমেন্টের কাজ

```
def add_numbers(x, y):  
    total = x + y  
    return total  
    print("This won't be printed")  
print(add_numbers(15, 25))
```

Output
40

Execution stops at return

return-এ ফাংশনের এক্সিকিউশন বন্ধ

Control goes back to caller

কন্ট্রোল কলিং ফাংশনে ফেরে

Can return value, variable, expression

মান, ভেরিয়েবল বা এক্সপ্রেশন: `return(x*y)`



1.4 Pass by Value vs Pass by Reference

পাস বাই ভ্যালু বনাম পাস বাই রেফারেন্স



Pass by Value

আরগুমেন্টের কপি তৈরি হয়। ফাংশনের ভেতরের পরিবর্তন মূল ভ্যালুতে পড়ে না। বেশি মেমরি ও সময় লাগে।



Pass by Reference

প্রকৃত ডাটার অ্যাড্রেস পাঠানো হয়। ভেতরের পরিবর্তন মূল ভ্যালুতে প্রতিফলিত হয়। কম মেমরি ও সময় লাগে।



How Python Actually Passes

পাইথনে বাস্তবে কীভাবে হয়



Immutable → acts like value

String, Integer, Tuple ফাংশনের ভেতরে পরিবর্তন করা যায় না



Mutable → acts like reference

List ফাংশনে পাস করলে ভেতরের পরিবর্তন কলিং ফাংশনেও প্রভাব ফেলে



Example 1: List Modified

উদাহরণ-১: রেফারেন্সের মতো আচরণ

```
def changeme(mylist):  
    print("This changes a passed list")  
    mylist.append([1, 2, 3, 4])  
    print("Inside:", mylist)  
    return
```

```
mylist = [10, 20, 30]  
changeme(mylist)  
print("Outside:", mylist)
```

Output

```
Inside: [10, 20, 30, [1, 2, 3, 4]]  
Outside: [10, 20, 30, [1, 2, 3, 4]]
```



Example 2: New Reference

উদাহরণ-২: ভ্যালুর মতো আচরণ

```
def changeme(mylist):  
    print("This changes a passed list")  
    mylist = [1, 2, 3, 4]  
    print("Inside:", mylist)  
    return
```

```
mylist = [10, 20, 30]  
changeme(mylist)  
print("Outside:", mylist)
```

Output

Inside: [1, 2, 3, 4]

Outside: [10, 20, 30]



Difference: Value vs Reference

পাস বাই ভ্যালু ও রেফারেন্সের পার্থক্য

Pass by Value / ভ্যালু	Pass by Reference / রেফারেন্স
Copies value to parameter / ভ্যালু কপি হয়	Passes the actual parameter / প্রকৃত প্যারামিটার পাস
Changes not reflected in original / মূল ভ্যালু বদলায় না	Changes reflected in original / মূল ভ্যালু বদলায়
Creates a copy / কপি তৈরি করে	Passes the address / অ্যাড্রেস পাস করে
More memory needed / বেশি মেমরি	Less memory needed / কম মেমরি
More time (copying) / বেশি সময়	Less time (no copy) / কম সময়



1.5 Parameter vs Argument

প্যারামিটার ও আরগুমেন্ট



Parameter (in definition)

ফাংশন ডিফাইনের বন্ধনীর ভেতরের ভেরিয়েবল, কমা দিয়ে আলাদা



Argument (in call)

ফাংশন কল করার সময় যে মান পাঠানো হয়

```
def add(a, b):      # a, b = parameters
    return a + b
add(5, 7)          # 5, 7 = arguments
```



4 Types of Arguments

আরগুমেন্টের ৪টি প্রকার



Required

রিকোয়ার্ড (পজিশনাল)



Keyword

কী-ওয়ার্ড আরগুমেন্ট



Default

ডিফল্ট আরগুমেন্ট



Variable length

*args ও **kwargs



Required Arguments

রিকোয়ার্ড আরগুমেন্ট

Missing argument → error কম আরগুমেন্ট দিলে error

```
def Number(x, y, z):  
    print(x, y, z)  
Number(x=10, y=20)
```

Output

```
TypeError: Number() missing 1 required  
positional argument: 'z'
```

All three given → works তিনটিই দিলে ঠিক

```
def Number(x, y, z):  
    print(x, y, z)  
Number(10, 20, 30)
```

Output

```
10 20 30
```



Keyword Arguments

কী-ওয়ার্ড আরগুমেন্ট

```
def add(x, y, z):  
    return x + y + z  
print(add(x=2, z=4, y=3))
```

Output
9

Values passed by name, not position

পজিশন নয়, নাম দিয়ে মান পাস

Order does not matter

ধারাবাহিকতা রক্ষা করতে হয় না



Default Arguments

ডিফল্ট আরগুমেন্ট

Example 1 উদাহরণ-১

```
def Number(x, y=30, z=20):  
    print(x, y, z)  
Number(10)
```

Output
10 30 20

Example 2: override উদাহরণ-২: মান বদল

```
def Number(x, y=30, z=20):  
    print(x, y, z)  
Number(50, 60, 70)
```

Output
50 60 70

Rule: default parameters must come from the right side | ডিফল্ট প্যারামিটার ডানদিক থেকে শুরু হবে



Variable Length Arguments

ভেরিয়েবল লেংথ আরগুমেন্ট

***args** → tuple নন কী-ওয়ার্ড

```
def Show(*Numbers):  
    print(type(Numbers))  
    print(Numbers)  
Show(50, 60, 70, 80, 90)
```

Output

```
<class 'tuple'>  
(50, 60, 70, 80, 90)
```

****kwargs** → dict কী-ওয়ার্ড

```
def Show(**Numbers):  
    print(type(Numbers))  
    print(Numbers)  
Show(x=50, y=60, z=70)
```

Output

```
<class 'dict'>  
{'x': 50, 'y': 60, 'z': 70}
```



1.6 Date & Time Modules

তারিখ ও সময় ফাংশন: ৫টি মডিউল

Module	Use / বিবরণ
tzinfo	Work with a time zone / টাইম জোন নিয়ে কাজ
date	Month, day, year only / শুধু তারিখ
datetime	Date + time together / তারিখ ও সময় একত্রে
time	Hour, minute, second, microsecond / সময়
timedelta	Duration of time / নির্দিষ্ট ডিউরেশন



time Module: Current Time

টাইম মডিউল: বর্তমান সময়

```
import time
localtime = time.localtime(time.time())
print("Local current time is:", localtime)
```

Output

```
Local current time is: time.struct_time(tm_year=2023, tm_mon=8, tm_mday=31, tm_hour=4,
tm_min=21, tm_sec=0, tm_wday=6, tm_yday=212, tm_isdst=0)
```



Methods of time Module

টাইম মডিউলের মেথডসমূহ

Method	Description
<code>time.tzset()</code>	Resets time conversion rules
<code>time.asctime([tupletime])</code>	Time-tuple to readable 24-char string
<code>time.ctime([secs])</code>	Like <code>asctime(localtime(secs))</code>
<code>time.gmtime([secs])</code>	Seconds since epoch to UTC time-tuple
<code>time.localtime([secs])</code>	Seconds since epoch to local time-tuple
<code>time.mktime(tupletime)</code>	Local time-tuple to seconds since epoch
<code>time.altzone</code>	Offset of local DST timezone (secs west of UTC)
<code>time.sleep(secs)</code>	Suspends calling thread for secs seconds
<code>time.strftime(fmt[,tupletime])</code>	Time-tuple to string by format fmt
<code>time.strptime(str,fmt)</code>	Parses string to time-tuple
<code>time.clock()</code>	CPU time in seconds (old Python)
<code>time.time()</code>	Current time in seconds since epoch



date Module: Today, Yesterday, Tomorrow

তারিখ মডিউল: আজ, গতকাল, আগামীকাল

Today আজকের তারিখ

```
from datetime import date
today = date.today()
print(today)
```

Output
2023-08-31

Yesterday গতকাল

```
from datetime import date,
timedelta
def yesterday():
    t = date.today()
    return t -
timedelta(days=1)
print(yesterday())
```

Output
2023-08-30

Tomorrow আগামীকাল

```
from datetime import date,
timedelta
def tomorrow():
    return date.today() +
timedelta(days=1)
print(tomorrow())
```

Output
2023-09-01



datetime Module

ডেইট টাইম মডিউল

```
import datetime
n = datetime.datetime.now()
t = datetime.datetime.today()
u = datetime.datetime.utcnow()
print("Now:", n)
print("UTC now:", u)
```

Output

```
Now: 2023-08-31 12:33:41.369818
UTC now: 2023-08-31 06:33:41.369837
```

```
import datetime
t = datetime.time(12, 46, 30)
d = datetime.date.today()
dt = datetime.datetime.combine(d, t)
print(dt)
```

Output

```
2023-08-31 12:46:30
```



timedelta & tzinfo

টাইম ডেল্টা ও টাইম জোন

timedelta সময়ের পার্থক্য

```
from datetime import timedelta
a = timedelta(days=1)
b = timedelta(hours=1)
c = a - b
print(c)
```

Output
23:00:00

tzinfo টাইম জোন

```
import datetime
plus6 = datetime.timezone(
    datetime.timedelta(hours=6))
d = datetime.datetime.now(plus6)
print(plus6, ":", d)
```

Output
UTC+06:00 : 2023-08-31 12:07:50+06:00



1.7 Program 1 & 2

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

1. Rectangle Area আয়তক্ষেত্রের ক্ষেত্রফল

```
def Area():  
    l = float(input("Length: "))  
    w = float(input("Width: "))  
    print("Area:", l * w)  
Area()
```

Output

```
Length: 8.5  
Width: 5.5  
Area: 46.75
```

2. Largest of Two বৃহত্তম সংখ্যা

```
def Largest():  
    a = int(input("1st: "))  
    b = int(input("2nd: "))  
    if a < b:  
        print("Largest:", b)  
    else:  
        print("Largest:", a)  
Largest()
```

Output

```
1st: 586  
2nd: 985  
Largest: 985
```



1.7 Program 3 & 4

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

3. Product of Numbers কতগুলো সংখ্যার গুণফল

```
def multiply(numbers):  
    result = 1  
    for x in numbers:  
        result = result * x  
    return result  
print(multiply((8, 2, 3, 12, 5)))
```

Output
2880

4. Factorial (Recursion) ফ্যাক্টোরিয়াল

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)  
n = int(input("Enter: "))  
if n < 0:  
    print("Negative, try again!")  
else:  
    print(n, "! =", factorial(n))
```

Output
Enter: 7
7 ! = 5040



1.7 Program 5 & 6

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

5. Prime Check মৌলিক সংখ্যা যাচাই

```
def Test_Prime(n):
    if n == 1:
        return 0
    elif n == 2:
        return 1
    else:
        for x in range(2, n):
            if n % x == 0:
                return 0
        return 1
n = int(input("Number: "))
print("Prime" if Test_Prime(n) else "Not prime")
```

Output

```
Number: 59
Prime
```

6. Reverse a String স্ট্রিং রিভার্স

```
def str_rev(s):
    r = ""
    index = len(s)
    while index > 0:
        r += s[index - 1]
        index = index - 1
    return r
print(str_rev(input("String: ")))
```

Output

```
String: python
nohtyp
```




1.7 Program 7 & 8

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

7. Triangle Area ত্রিভুজের ক্ষেত্রফল

```
import math
def Triangle():
    a = int(input("a: "))
    b = int(input("b: "))
    c = int(input("c: "))
    if (a+b)>c and (b+c)>a and (a+c)>b:
        s = (a + b + c) / 2
        print(math.sqrt(s*(s-a)*(s-b)*(s-c)))
    else:
        print("Triangle is not possible")
Triangle()
```

Output

```
a: 3 b: 4 c: 5
6.0
```

8. Circle Area বৃত্তের ক্ষেত্রফল

```
import math
def CircleArea():
    r = float(input("Radius: "))
    print("Area:", math.pi * r ** 2)
CircleArea()
```

Output

```
Radius: 5
Area: 78.53981633974483
```



1.7 Program 9 & 10

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

9. Sum of Odd (1-100) বিজোড় সংখ্যার যোগফল

```
def summation():
    total = 0
    for n in range(100):
        if n % 2 == 0:
            continue
        total += n
    return total
print("Summation:", summation())
```

Output

Summation: 2500

10. Fibonacci Series ফিবোনাচ্চি সিরিজ

```
def fibo(n):
    if n <= 1:
        return n
    else:
        return fibo(n-1) + fibo(n-2)
for i in range(10):
    print(fibo(i), end=" ")
```

Output

0 1 1 2 3 5 8 13 21 34



1.7 Program 11 & 12

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

11. Sum of 5 List Numbers লিস্টের ৫টি সংখ্যার যোগ

```
def add_numbers():
    numbers = []
    for i in range(5):
        num = float(input("Number: "))
        numbers.append(num)
    print("Sum:", sum(numbers))
add_numbers()
```

Output

```
(inputs 22, 36, 56, 98, 12)
Sum: 224.0
```

12. Count Digits ডিজিট গণনা

```
def count_digits(n):
    count = 0
    while n > 0:
        n //= 10
        count += 1
    return count
num = int(input("Number: "))
print("Total digits:", count_digits(num))
```

Output

```
Number: 1234
Total digits: 4
```



1.7 Program 13 & 14

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

13. Palindrome Check প্যালিনড্রোম যাচাই

```
def is_palindrome(word):  
    rev = ""  
    for ch in word:  
        rev = ch + rev  
    return word == rev  
w = input("Word: ")  
if is_palindrome(w):  
    print("Palindrome")  
else:  
    print("Not Palindrome")
```

Output

```
Word: madam  
Palindrome
```

14. Even Numbers from List লিস্ট থেকে জোড় সংখ্যা

```
def extract_even(lst):  
    evens = []  
    for n in lst:  
        if n % 2 == 0:  
            evens.append(n)  
    return evens  
nums = [1, 2, 3, 4, 5, 6, 7, 8]  
print("Even:", extract_even(nums))
```

Output

```
Even: [2, 4, 6, 8]
```



1.7 Program 15

ইউজার ডিফাইন ফাংশন দিয়ে প্রোগ্রাম

15. Perfect Number পারফেক্ট নাম্বার (৬ = ১+২+৩)

```
def is_perfect(n):  
    total = 0  
    for i in range(1, n):  
        if n % i == 0:  
            total += i  
    return total == n  
num = int(input("Number: "))  
print("Perfect" if is_perfect(num) else "Not  
Perfect")
```

Output

```
Number: 6  
Perfect
```



Chapter Recap

অধ্যায়ের সারসংক্ষেপ



def name(params):

ফাংশন তৈরি: def, নাম, কোলন



return

মান ফেরত; না থাকলে None



Mutable vs Immutable

List বদলায়, String/Tuple বদলায় না



4 argument types

Required, Keyword, Default,
*args/**kwargs



time / datetime

তারিখ ও সময় ম্যানেজ করা



Practice

১৫টি প্রোগ্রাম নিজে করা



Class Activity

ক্লাস অ্যাক্টিভিটি: নিজে করো



1. `cube(x)`

একটি সংখ্যার ঘন (cube) রিটার্ন করো



2. `greet(name="Student")`

ডিফল্ট আরগুমেন্টসহ শুভেচ্ছা ফাংশন



3. `average(*nums)`

যেকোনো সংখ্যক সংখ্যার গড় বের করো



4. `days_left(date)`

timedelta দিয়ে আজ থেকে একটি তারিখ পর্যন্ত দিন